

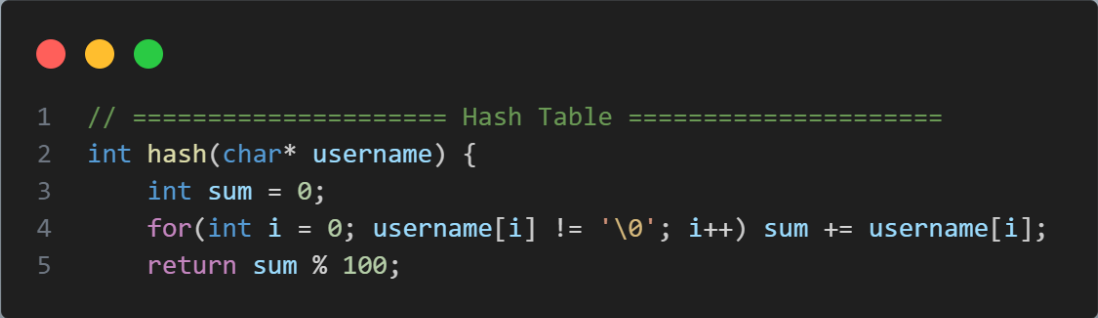
Penggunaan Data Structure dalam Warehouse Management

1. Hash Table

Penggunaan Hash digunakan untuk membuat akses yang cepat. Sistem akan memverifikasi user dan password.

Cara Kerja dan Penggunaannya

- Setiap elemen dalam array merupakan head dari sebuah linked list. Metode ini merupakan separate chaining yang berfungsi untuk menangani collision.
- Pada function hash, hash digunakan untuk mengubah username menjadi index array. Hasilnya merupakan angka 0-99



```
1 // ===== Hash Table =====
2 int hash(char* username) {
3     int sum = 0;
4     for(int i = 0; username[i] != '\0'; i++) sum += username[i];
5     return sum % 100;
```

- Inserthashtotable merupakan fungsi selanjutnya, fungsi ini berfungsi untuk saat register atau load dari file, user yang register tersebut akan dimasukkan ke dalam hash table. User yang baru akan ditaruh pada linked list yang paling depan pada bucket hash

```

1 void insertUserToHashTable(const char *userID, const char *username, const char *password) {
2     int idx = hash((char*)username);
3     User *newUser = (User*)malloc(sizeof(User));
4     strcpy(newUser->userID, userID);
5     strcpy(newUser->username, username);
6     strcpy(newUser->password, password);
7     newUser->next = head[idx];
8     head[idx] = newUser;
9 }

```

- Pencarian user(login)

Saat login, username di-hash untuk dapatkan index bucket.

Linked list pada bucket itu di-traverse untuk mencari username yang cocok

```

1 int index = hash(username);
2 User* curr = head[index];
3 while(curr) {
4     if(strcmp(username, curr->username) == 0) {
5         // ditemukan
6     }
7     curr = curr->next;
8 }

```

Alur dari penggunaan hash adalah user yang melakukan register, registerasinya akan masuk ke dalam hashtable dengan fungsi hash dan linked list. Lalu ketiga login username akan di hash dan di cari linked list bucket itu. Password yang ditemukan akan dicek setelah username ditemukan. Dengan menggunakan *hash table*, pencarian *username* memiliki kompleksitas waktu rata-rata $O(1)$, jauh lebih cepat daripada harus menelusuri seluruh daftar pengguna satu per satu ($O(n)$).

2. Linked List

Linked list digunakan untuk menyimpan data dari pointer ke node berikutnya. Jumlah elemennya yang dinamis. Dalam penerapan linked list ada beberapa contoh penerapannya.

Struct Warehouse

Di dalam kode, struct Warehouse terdapat pointer bernama struct Warehouse *next. Kode ini berfungsi untuk menunjuk ke warehouse berikutnya dalam daftar, sehingga seluruh data warehouse disusun dalam bentuk linked list. Penggunaan linked list ini bertujuan agar data warehouse dapat dikelola dengan mudah, sehingga warehouse bisa ditambah atau dihapus tanpa perlu menggeser data lain seperti pada array. Dengan linked list, penambahan atau penghapusan warehouse cukup dengan mengatur pointer next, sehingga prosesnya efisien dan fleksibel. Selain itu, linked list juga memungkinkan jumlah warehouse yang disimpan dapat bertambah atau berkurang secara dinamis sesuai kebutuhan program, tanpa perlu menentukan kapasitas maksimum di awal.



```
1 typedef struct Warehouse {
2     char warehouseID[8];
3     char whname[101];
4     char whlocation[101];
5     int whcapacity;
6     struct Warehouse *next;
7 } Warehouse;
```

Struct Item

Pada struct Item terdapat pointer struct Item *next yang membentuk sebuah linked list, yaitu daftar seluruh item yang ada di sistem. Fungsi utama penggunaan linked list pada item adalah untuk menyimpan, menambah, mengubah, dan menghapus

data barang secara dinamis. Saat menambah barang baru (fungsi `addItem` atau `addItemToWarehouse`), node item baru akan langsung dihubungkan ke awal linked list tanpa perlu menggeser data lain. Saat mengubah data barang (fungsi `updateItem`), program cukup mencari node yang sesuai lalu mengedit datanya. Untuk menghapus barang (fungsi `deleteItem`), node yang dihapus cukup dilepas dari rantai linked list dan memorinya dibebaskan, tanpa perlu menggeser elemen lain seperti pada array.

Penggunaan linked list pada item membuat pengelolaan data barang menjadi lebih efisien dan fleksibel, terutama jika jumlah barang sering berubah-ubah. Selain itu, linked list memungkinkan penambahan dan penghapusan data dilakukan dengan kompleksitas waktu yang rendah, serta tidak membutuhkan alokasi ulang memori dalam jumlah besar seperti pada array.

```
1  typedef struct Item {
2      char itemID[8];
3      char warehouseID[8];
4      char itemName[101];
5      char itemCategory[101];
6      char itemWeight[20];
7      int itemQuantity;
8      char itemDestination[101];
9      int itemPriority;
10     struct Item *next;
11 } Item;
```

Struct User

Linked list pada item digunakan untuk menyimpan daftar seluruh barang secara dinamis. Penambahan, pengubahan, dan penghapusan item dapat dilakukan dengan mudah tanpa perlu menggeser data lain.

```
1  typedef struct User {
2      char userID[10];
3      char username[101];
4      char password[101];
5      struct User* next; // pointer ke user berikutnya
6 } User;
7
```

3. Binary Search Tree

Struktur data pohon di mana untuk setiap *node*, semua nilai di sub-pohon kiri lebih kecil daripada nilai *node* tersebut, dan semua nilai di sub-pohon kanan lebih besar. Implementasi dari BST digunakan untuk melakukan search item dengan nama.

Cara kerja BST :

1. Program akan membuat BST sementara untuk item-item yang ada menggunakan fungsi insertBST.
2. Melakukan pencarian dengan fungsi searchBST, sehingga pencarian dilakukan lebih cepat.
3. Pencarian yang telah selesai, memori pada BST akan dikosongkan dengan freeBST sehingga BST tidak menyimpan permanen di memori.

BST diimplementasikan dengan struct yang berisi pointer ke Item dan dua anak (left, right). Pohon diurutkan berdasarkan itemName secara alfabetis. Waktu yang dibutuhkan untuk melakukan pencarian sangat efisien yang berada pada $O(\log n)$

Kode BST

```
1 struct BSTNode {
2     Item *item;
3     struct BSTNode *left, *right;
4 };
5
6 BSTNode* insertBST(BSTNode* root, Item* item) {
7     if (!root) {
8         BSTNode* node = (BSTNode*)malloc(sizeof(BSTNode));
9         node->item = item; node->left = node->right = NULL;
10        return node;
11    }
12    if (strcmp(item->itemName, root->item->itemName) < 0)
13        root->left = insertBST(root->left, item);
14    else
15        root->right = insertBST(root->right, item);
16    return root;
17 }
18
19 Item* searchBST(BSTNode* root, const char* name) {
20     if (!root) return NULL;
21     int cmp = strcmp(name, root->item->itemName);
22     if (cmp == 0) return root->item;
23     if (cmp < 0) return searchBST(root->left, name);
24     return searchBST(root->right, name);
25 }
26
27 void freeBST(BSTNode* root) {
28     if (!root) return;
29     freeBST(root->left);
30     freeBST(root->right);
31     free(root);
32 }
```

4. Heap

Heap digunakan untuk mengeluarkan barang prioritas pada warehouse. Barang yang memiliki prioritas tertinggi akan berada pada paling atas (root). Heap yang digunakan adalah heap max. Heap max karena pada fungsi heapifyUp dan heapifyDown terdapat

```
heap[idx]->itemPriority > heap[(idx-1)/2]->itemPriority
```

dan

```
if (heap[l]->itemPriority > heap[largest]->itemPriority) largest = l;
```

```
if (heap[r]->itemPriority > heap[largest]->itemPriority) largest = r;
```

Berarti nilai itemPriority terbesar akan selalu berada di root (indeks 0)



Heap disini merupakan pointer untuk item dan heapzise menyimpan jumlah elemen heap pada saat ini

Terdapat Fungsi yang digunakan dalam heap

pushHeap adalah fungsi untuk menambahkan item baru ke dalam heap. Setelah item dimasukkan ke posisi paling akhir pada array heap, fungsi ini akan memanggil heapifyUp untuk menata ulang posisi item tersebut ke atas, sehingga aturan max-heap tetap terjaga—yaitu, setiap parent memiliki nilai prioritas lebih besar daripada child-nya.

popHeap digunakan untuk mengambil dan menghapus item dengan prioritas tertinggi dari heap. Fungsi ini akan mengambil item pada posisi root (indeks 0), lalu menggantinya dengan item terakhir di heap. Setelah itu, heapifyDown dipanggil untuk menata ulang heap dari atas ke bawah agar struktur max-heap tetap valid.

heapifyUp dan **heapifyDown** adalah dua fungsi penting untuk menjaga properti max-heap setelah terjadi penambahan atau penghapusan item. **heapifyUp** digunakan saat penambahan item, menukar posisi item ke atas jika prioritasnya lebih tinggi dari parent-nya. Sebaliknya, **heapifyDown** digunakan setelah penghapusan root, menukar posisi item ke bawah jika ada child dengan prioritas lebih tinggi, sehingga heap tetap terurut dengan benar.

```
1 void heapifyUp(int idx) {
2     while (idx > 0 && heap[idx]->itemPriority > heap[(idx-1)/2]->itemPriority) {
3         Item* tmp = heap[idx];
4         heap[idx] = heap[(idx-1)/2];
5         heap[(idx-1)/2] = tmp;
6         idx = (idx-1)/2;
7     }
8 }
9 void heapifyDown(int idx) {
10     int largest = idx, l = 2*idx+1, r = 2*idx+2;
11     if (l < heapSize && heap[l]->itemPriority > heap[largest]->itemPriority) largest = l;
12     if (r < heapSize && heap[r]->itemPriority > heap[largest]->itemPriority) largest = r;
13     if (largest != idx) {
14         Item* tmp = heap[idx]; heap[idx] = heap[largest]; heap[largest] = tmp;
15         heapifyDown(largest);
16     }
17 }
18 void pushHeap(Item* item) {
19     heap[heapSize] = item;
20     heapifyUp(heapSize++);
21 }
22 Item* popHeap() {
23     if (heapSize == 0) return NULL;
24     Item* top = heap[0];
25     heap[0] = heap[--heapSize];
26     heapifyDown(0);
27     return top;
28 }
29
```

5. Queue

initQueue

Fungsi ini menginisialisasi queue dengan mengatur pointer front dan rear menjadi NULL, menandakan queue masih kosong.

enqueue

Fungsi ini menambahkan item ke dalam queue. Node baru dialokasikan dan diisi dengan pointer ke item. Jika queue kosong (`rear == NULL`), node baru menjadi front dan rear. Jika tidak, node baru ditambahkan di belakang (`rear->next`), lalu rear di-update ke node baru.

dequeue

Fungsi ini mengeluarkan item dari queue. Node paling depan (front) diambil, pointer front digeser ke node berikutnya, dan node lama dibebaskan dari memori. Jika

setelah pengambilan queue menjadi kosong, rear juga di-set ke NULL. Fungsi ini mengembalikan pointer ke item yang dikeluarkan.

Queue ini digunakan untuk mengeluarkan barang dari warehouse secara FIFO (First-In-First-Out)

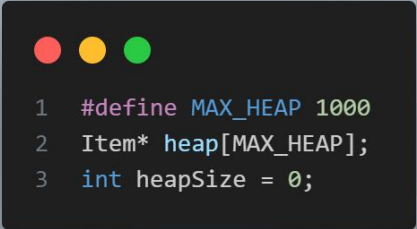
Kode Queue

```
1 void initQueue(Queue *q) { q->front = q->rear = NULL; }
2
3 void enqueue(Queue *q, Item *item) {
4     QueueNode *node = (QueueNode*)malloc(sizeof(QueueNode));
5     node->item = item; node->next = NULL;
6     if (!q->rear) q->front = q->rear = node;
7     else { q->rear->next = node; q->rear = node; }
8 }
9 Item* dequeue(Queue *q) {
10     if (!q->front) return NULL;
11     QueueNode *tmp = q->front;
12     Item *item = tmp->item;
13     q->front = q->front->next;
14     if (!q->front) q->rear = NULL;
15     free(tmp);
16     return item;
```

6. Array

Array digunakan untuk menyimpan data pada heap, hash, buffer, dan input buffer

Array pada heap digunakan untuk menyimpan pointer ke item untuk fitur "Keluarkan barang prioritas"



```
1 #define MAX_HEAP 1000
2 Item* heap[MAX_HEAP];
3 int heapSize = 0;
```

Array pada hash digunakan untuk setiap elemen array adalah head dari linked list user pada bucket hash tertentu.

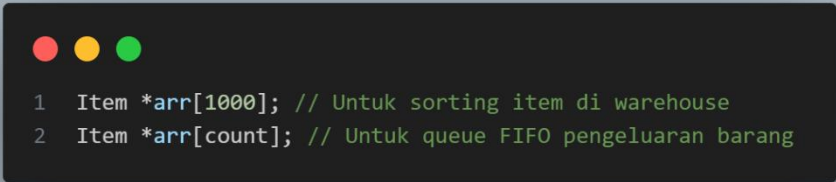
Kode Hash



```
1 User* head[100];
```

Array pada buffer sementara berfungsi untuk menampung pointer ke item sebelum diproses lebih lanjut (misal: sorting dengan bubble sort, atau enqueue ke queue).

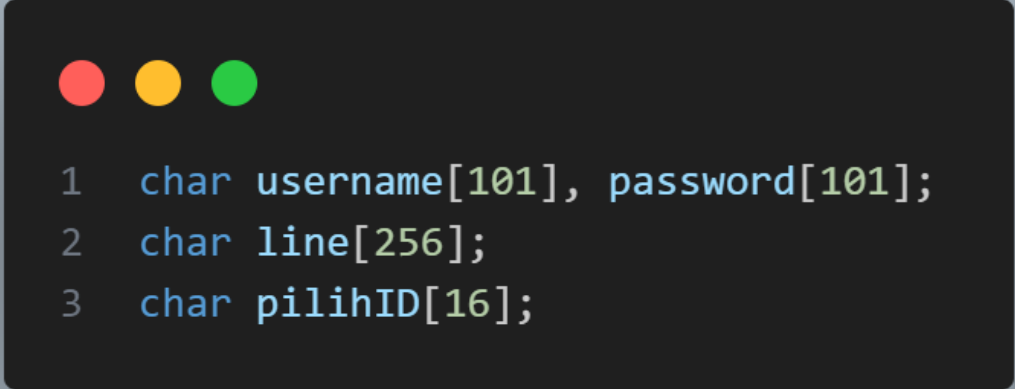
Kode Buffer



```
1 Item *arr[1000]; // Untuk sorting item di warehouse
2 Item *arr[count]; // Untuk queue FIFO pengeluaran barang
```

Array pada input buffer menyimpan data input sementara sebelum diproses.

Kode Input Buffer



```
1 char username[101], password[101];  
2 char line[256];  
3 char pilihID[16];
```

Array digunakan untuk mengimplementasikan heap (priority queue) pada proses pengeluaran barang prioritas, sebagai tabel hash untuk manajemen user, serta sebagai buffer sementara saat melakukan sorting dan proses queue FIFO. Selain itu, array juga digunakan untuk menyimpan input dari user dan data sementara lainnya. Dengan memanfaatkan array, aplikasi dapat melakukan akses data secara cepat dan efisien, terutama pada operasi yang membutuhkan akses indeks langsung seperti heap, hash table, dan sorting